

Plato’s Cave: A Human-Centered Research Verification System

Anonymous submission

Abstract

The exponential growth in research publications has created an urgent need for better ways to fact-check information, assess writing quality, and identify unverifiable claims. We present Plato’s Cave as a human-centered research verification system that (i) extracts directed acyclic graph (DAG) from a body of text to represent claims and logical arguments and (ii) assigns credibility scores to nodes and edges from the DAG as a means to interpret and evaluate papers’ argumentative structures. Our current implementation uses a factorized pipeline where one LLM samples multiple candidate DAGs (roles and dependencies), a second LLM produces verification metrics for each node, and a graph scorer aggregates these signals via role priors, trust-gated propagation, and path-based objectives. We report the system implementation, as well as preliminary results on a collected dataset of 104 research papers.

1 Introduction

Scientific publishing continues to scale faster than traditional review capacity. In 2022, there were 3.3 million research publications in science and engineering (National Science Board 2023). This proliferation has strained traditional peer review mechanisms and allowed the spread of unreliable research. The total number of articles in the Retraction Watch Database increased 26% in 2025, now exceeding 63,000, and high-profile cases of fabricated data have undermined confidence in published findings (Retraction Watch 2024, 2025). This situation urges the development of tools that can help researchers and reviewers rapidly audit the internal logic of papers as structured arguments where claims, methods, evidence, and conclusions are complex and interconnected.

Existing approaches towards research verification fall into three categories. Citation-based metrics such as the h-index and impact factors of the journals measure popularity rather than correctness and may distort performance targets that should prioritize logic (Strathern 1997). Fact-checking systems such as SciFact (Wadden et al. 2020) and FEVER (Thorne et al. 2018) operate at the sentence level and fail to capture the argumentative structure that connects claims to evidence. Lastly, approaches to pure language models lack an in-depth knowledge of external sources and cannot reliably verify claims against primary sources (Liu and Shah 2023).

We address these limitations by posing a system in which scientific papers are modeled as knowledge graphs, where

nodes represent semantic units (hypotheses, claims, evidence, methods, results), and edges encode their associated logical relationships. Our system, Plato’s Cave, extracts these graphs and verifies individual claims using autonomous browser-use agents that search the web to assess source credibility by extracting supporting or contradicting evidence. The verified node qualities propagate through the graph structure via a novel trust-gating mechanism: when a parent node has low trust, it exponentially reduces confidence in all dependent child nodes, ensuring that weak foundational claims appropriately diminish confidence in their conclusions. This propagation step is inspired by message-passing inference in probabilistic graphical models (i.e. belief propagation / sum-product), but here it propagates verification-derived trust rather than probabilistic marginals. (Pearl 1988; Kschischang, Frey, and Loeliger 2001; Yedidia, Freeman, and Weiss 2003; Wainwright and Jordan 2008). Our contributions include:

- A practical, role-aware DAG representation for papers together with a robust validation-and-repair step that enforces ontology and acyclicity constraints.
- A multi-level scoring framework that combines node qualities, edge-level priors and semantic alignment, and global structure metrics (bridge coverage, best-path reliability, redundancy, fragility, coherence, and coverage).
- An efficient factorized sampling scheme that separately perturbs structure (K sampled DAGs) and aggregation parameters (M resampled node/propagation weights), enabling uncertainty estimates without retraining.
- A pilot engineering evaluation in 149 records (104 PDFs available) showing end-to-end throughput and identifying current failure modes (e.g., invalid role labels) and calibration gaps between scores and coarse spreadsheet ratings.
- Visual representations of papers to easily summarize decisions in the auditing system to aid in rapid validation

The remainder of this paper proceeds as follows. Section 2 reviews related work in claim verification, knowledge graphs, and trust propagation. Section 3 formalizes the paper verification problem. Section 4 describes our approach including the knowledge graph ontology, multi-agent verification pipeline, and tech stack. Section 5 presents experimental results comparing our system with baselines. Section 6 discusses limitations and future work.

2 Related Work

Research verification intersects several strands of prior work, spanning scientific claim verification, knowledge graph construction, trust propagation, and emerging multi-agent systems for automated information gathering. We structure this section around these themes to situate Plato’s Cave within the broader landscape: beginning with scientific claim verification, moving through graph-based representations and trust-propagation methods, and concluding with multi-agent systems and our positioning relative to these approaches.

Scientific Claim Verification

Automated fact-checking has received significant attention in natural language processing. The FEVER dataset introduced the task of verifying claims against Wikipedia (Thorne et al. 2018). The study has led to spawning numerous neural architectures for evidence retrieval and entailment classification. SciFact (Wadden et al. 2020) adapted this paradigm to scientific claims, pairing assertions with relevant abstracts and stance labels (i.e. supports, refutes, not_enough_info). However, these systems operate at the sentence level and treat verification as an isolated classification task rather than modeling the argumentative structure of complete papers.

Other studies have explored claim decomposition (Liu, Datta, and Lim 2015) and multi-hop reasoning (Yang et al. 2018), but these approaches still focus on question-answering rather than holistic paper assessment. Our work differs by extracting the complete dependency graph of a paper’s arguments and propagating trust through this structure rather than verifying claims independently.

Knowledge Graph Construction

Automated knowledge graph construction from text has progressed through rule-based extraction (Cheng et al. 2024), supervised relation extraction (Liu 2020), and more recently, LLM-based approaches (Zhu et al. 2024). Scientific knowledge graphs like Semantic Scholar’s API (Kinney et al. 2023) and Open Academic Graph (Zhang et al. 2019), with the latter focusing on unifying the Microsoft Academic Graph (Sinha et al. 2015) and AMiner (Tang et al. 2008) focus on citation networks and metadata (such as authors, institutions, field of study) rather than the internal argumentative structure of papers.

Argumentation mining (Lawrence and Reed 2019) seeks to identify claim-premise relationships in text, but existing approaches focus on pre-defined structures, such as internet forums or student essays. Our work extends these ideas to scientific papers, introducing a domain-specific ontology of 10 semantic roles and enforcing structural constraints like acyclicity and role-specific parent-child relationships.

Trust Propagation

In probabilistic graphical models, belief propagation (BP) is a canonical message-passing algorithm for propagating local evidence through a graph to compute marginal beliefs, with the sum-product algorithm providing a unifying view on factor graphs. (Pearl 1988; Kschischang, Frey, and Loeliger 2001) Loopy and generalized variants connect BP

fixed points to variational objectives (e.g., Bethe-style free energies), providing a principled lens on approximate propagation in graphs with complex dependencies. (Yedidia, Freeman, and Weiss 2003; Wainwright and Jordan 2008)

Trust propagation has been widely studied in social network analysis (Guha et al. 2004) and citation networks (Brin and Page 1998; Kleinberg 1999). These approaches model trust as flowing through edges weighted by structural properties (citation count, user ratings). However, they lack grounding in content-based verification of the actual claims being made, and something we aim to include to ensure automated validity of the claim.

Recent work on misinformation detection (Mridha et al. 2021) has incorporated source credibility and cross-source consistency, but these methods aggregate signals at the document level rather than modeling fine-grained dependency chains within a single paper’s argumentation. Our trust-gating mechanism differs by explicitly modeling how parent node quality gates the confidence of child nodes, preventing weak evidence from inflating downstream claims through a differentiable threshold function.

Multi-Agent Systems

Autonomous agents with web-browsing capabilities have emerged as a powerful paradigm for information gathering (Nakano et al. 2021; Browser-Use 2024). Recent systems combine large language models with browser automation to answer questions, compare products, and fact-check claims. Our work applies this paradigm to scientific verification, coordinating multiple specialized agents to extract verification metrics across six dimensions (credibility, relevance, evidence strength, method rigor, reproducibility, citation support).

Positioning this Work in the Broader Literature

We uniquely combine knowledge graph construction with content-based multi-agent verification and trust propagation in a learnable framework. We aim to expand fact checking models from sentence-level to complete argumentative structures. Unlike citation networks, we want to ensure this fact checking is based in verified content rather than popularity. We also strive to use LLMs to help deploy agents to find external sources rather than using the LLM itself. These factors allow us to create a system that provides interpretable assessments of a paper’s quality with existing performance baselines.

3 Problem Formulation

We formalize the scoring problem as follows. Given a candidate knowledge graph extracted from a paper (nodes with roles and text, and directed dependency edges) together with per-node verification metrics, our goal is to compute calibrated node qualities, edge confidences, and an overall graph score. Formally, we represent the paper as a knowledge graph $G = (V, E)$ where:

- Each node $v \in V$ represents a semantic unit with role $\rho_v \in \mathcal{R}$ and quality score $q_v \in [0, 1]$.

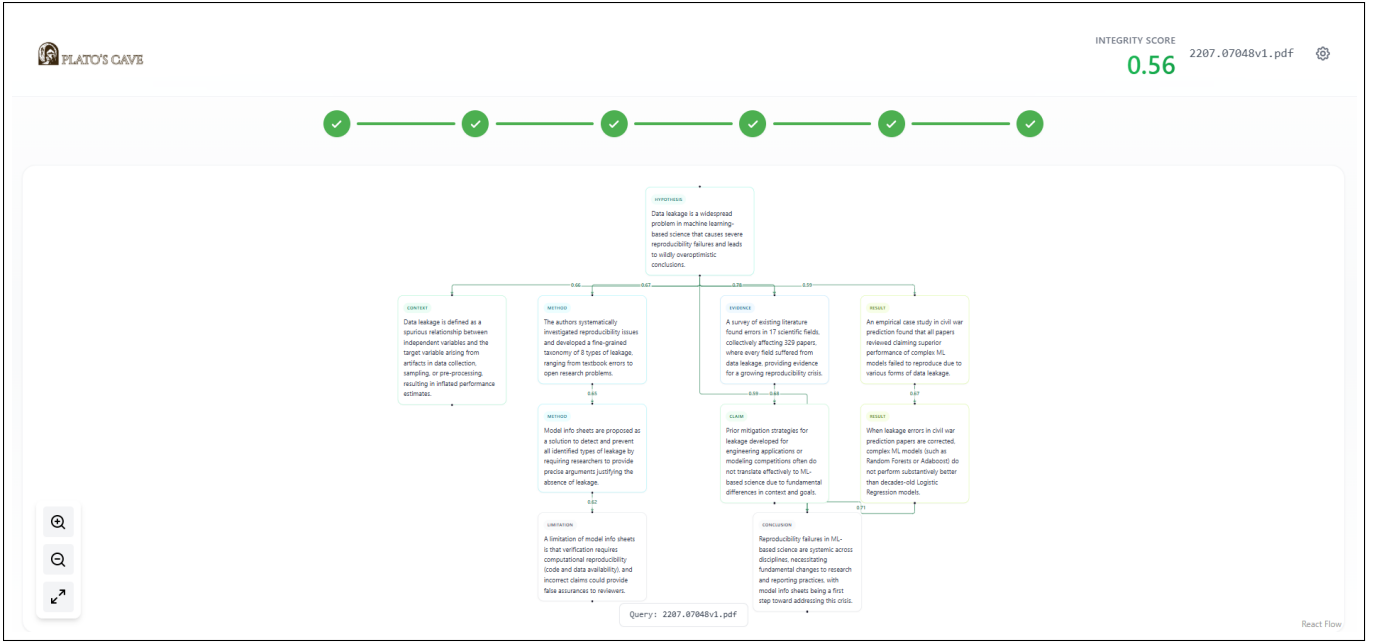


Figure 1: Finalized run in the deployed system with the visualized DAG and Integrity Score

- Each edge $(u, v) \in E$ represents a logical dependency with confidence $C_{u \rightarrow v} \in [0, 1]$.
- The graph $\mathcal{G}$ is a directed acyclic graph (DAG) respecting role-specific constraints.

The semantic role set is defined as $\mathcal{R} := \{ \text{Assumption, Claim, Conclusion, Context, Counterevidence, Evidence, Hypothesis, Limitation, Method, Result} \}$ following the scorer’s canonical role ontology. From $\mathcal{G}$, we compute an overall graph score $S_{\text{graph}} \in [0, 1]$ that aggregates six interpretable components $\mathcal{T} := \{ \text{bridge coverage, best-path reliability, redundancy, fragility, coherence, and coverage} \}$. Our objective is for S_{graph} to correlate strongly with human expert assessments of paper quality, where:

The coefficients $\{\gamma_t\}_{t \in \mathcal{T}}$ are nonnegative component weights (higher γ_t indicates greater contribution of component t to the aggregate); unless otherwise stated, we constrain $\sum_{t \in \mathcal{T}} \gamma_t = 1$ for interpretability.

$$S_{\text{graph}} = \phi_{\text{clip}[0,1]} \left(\sum_{t \in \mathcal{T}} \gamma_t S_t \right). \quad (1)$$

The key challenges in this problem formulation are listed below, and are approached in the following implementation:

1. **Structure extraction:** Parsing unstructured text into a structured DAG with correct semantic roles.
2. **Claim verification:** Assessing the validity of individual nodes using external knowledge.
3. **Trust propagation:** Propagating node qualities through edges via a BP-inspired message-passing scheme such that weak evidence does not inflate downstream claims. (Pearl 1988; Kschischang, Frey, and Loeliger 2001; Yedidia, Freeman, and Weiss 2003)

4. **Interpretability:** Ensuring scores are explainable and aligned with human reasoning.

4 Implementation

This section describes the reference implementation used for the attached experiments. The implementation is intentionally *factorized* into: structure extraction, node verification, and graph scoring are separate modules connected by simple JSON artifacts, enabling independent iteration and ablation.

System Overview

Given a PDF, the pipeline executes five stages, per Figure 2 and detailed in the following sections:

1. **Text extraction:** extract and lightly normalize text from the PDF using PyMuPDF (`fitz`).
2. **DAG generation:** generate and sample a role-aware DAG (nodes + directed dependencies) from the extracted text using an LLM.
3. **Node verification:** for each node, produce six normalized verification metrics in $[0, 1]$ using an LLM (optionally with lightweight retrieval).
4. **Graph scoring:** compute node qualities, trust-gated edge confidences, and a paper-level score from the DAG.
5. **Visualization:** The frontend renders the graph interactively with Dagre layout, displaying metric tooltips and highlighting high-confidence paths.

The experiments run this pipeline in parallel across papers and within a paper (bounded by explicit semaphores for node-level and global LLM concurrency), and write per-paper artifacts (sampled DAGs, per-trial scores, and summaries) for inspection and post-hoc analysis.

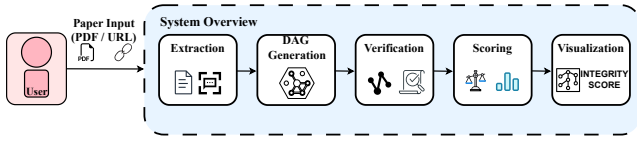


Figure 2: System Overview

Knowledge Graph Construction

We use a fixed ontology of 10 roles: (1) **Hypothesis**, (2) **Claim**, (3) **Evidence**, (4) **Method**, (5) **Result**, (6) **Conclusion**, (7) **Assumption**, (8) **Counterevidence**, (9) **Limitation** and (10) **Context**. Each node is a short excerpt of the paper (typically 1–3 sentences) labeled with one of these roles. Edges represent *dependency flow* from higher-level framing toward downstream synthesis; in the default priors, Hypothesis nodes are treated as roots and Conclusion nodes as sinks, with intermediate support roles (i.e. Evidence/Method/Result/Claim) on hypothesis to conclusion bridges.

A DAG extractor prompts an LLM with the paper text, the ontology, and a strict JSON schema (nodes with `id`, `role`, `text`, `parents`, `children`). The output is validated for: (i) ontology compliance, (i.e. the role of each node must be one of the 10 allowed values), (ii) referential integrity (i.e. parent/child IDs must exist), and (iii) acyclicity. If validation fails, the system attempts a bounded “repair” pass by returning the validation errors to the LLM and requesting a corrected JSON.

In the attached run, DAG extraction uses `gpt-5-mini` and enforces a maximum of 16 nodes per sampled DAG.

Node Verification

Node verification is performed by an ensemble of LLM verifiers, where this agentic system is made available in a corresponding code release to aid replication. In this released scorer, verifier agents designate the presence or absence of six metrics per node (i.e. credibility, relevance, evidence strength, method rigor, reproducibility, citation support), each in $[0, 1]$. Each verification agent is a browser automation instance powered by an LLM with access to web search, navigation, and content extraction tools. Agents use Exa as a neural search backend to retrieve web results for scientific queries, which are then assessed by the browser-use agent. Agents receive a node (including the role and associated text snippet) and execute a search-verify-assess workflow:

1. **Search**: Formulate search queries based on the node text and role (e.g., for Evidence nodes, search for the cited source; for Method nodes, search for validation studies).
2. **Navigate**: Click search results, scroll to relevant sections, extract text.
3. **Assess**: Evaluate source credibility (domain authority, publication venue), relevance (node claim alignment), and supporting evidence (data, citations).

We use the browser-use library (Browser-Use 2024), which optimizes LLM-browser interaction with action batching and caching, achieving 3-5x speedup over naive implementations.

Agents run in Docker containers with remote browsers accessed via Chrome DevTools Protocol, enabling visual monitoring through noVNC. For each node v , verification agents extract six metrics $\mathbf{m}_v = (m_{v,1}, \dots, m_{v,6}) \in [0, 1]^6$:

1. **Credibility** (m_1): Trustworthiness of sources (peer-reviewed journals score higher than blogs).
2. **Relevance** (m_2): Alignment between node claim and retrieved evidence.
3. **Evidence strength** (m_3): Quality and quantity of supporting data.
4. **Method rigor** (m_4): For Method nodes, soundness of experimental design.
5. **Reproducibility** (m_5): Availability of code, data, or detailed procedures.
6. **Citation support** (m_6): Number and quality of citations backing the claim.

The agent extracts these metrics by prompting the LLM with retrieved content and a structured output schema. Metrics are normalized to $[0, 1]$ and stored alongside the node.

Trust-Propagating Graph Scoring

This subsection summarizes the *real-time* scoring backend used to evaluate a candidate paper-DAG after node-level metrics are available. The goal is to assign (i) a **quality** to each node, (ii) a **confidence** to each dependency edge, and (iii) a **paper-level score** that reflects whether at least one hypothesis is supported by a coherent chain of evidence, method, and results.

Inputs and update protocol The scorer consumes a validated DAG $\mathcal{G} = (V, E)$ where each node $v \in V$ has: (i) role $\rho_v \in \mathcal{R}$, (ii) node text t_v , and (iii) metrics $\mathbf{m}_v \in [0, 1]^6$. Node qualities and edge confidences are recomputed after metrics are available for a node and its parents, ensuring stability of intermediate results.

Node quality (role-aware metric fusion) Each node’s metrics are optionally reweighted by a *global* per-metric weight vector, then fused into a single node quality score $q_v \in [0, 1]$ using *role-specific* weights. Weights are normalized internally (by ℓ_1 magnitude) and the resulting score is clipped to $[0, 1]$.

Edge confidence and trust propagation Each directed edge ($u \rightarrow v$) receives two related scores:

- **Raw edge confidence** $C_{u \rightarrow v}^{\text{raw}}$: a local plausibility score computed as a weighted mixture of: (i) a role-transition prior for (ρ_u, ρ_v) , (ii) parent and child node qualities (q_u, q_v) , (iii) a lightweight lexical alignment score between node texts (Jaccard overlap), and (iv) a role-pair *synergy* term that mixes parent and child metrics in a role-specific way.
- **Final edge confidence** $C_{u \rightarrow v}$: a trust-gated version of $C_{u \rightarrow v}^{\text{raw}}$ that down-weights dependencies coming from untrustworthy upstream nodes.

Trust propagation assigns each node a propagated trust value $t_v \in [0, 1]$ that combines its own quality q_v with the “weakest-link” character of upstream support. Each parent

contributes an amount proportional to its trust (raised to an exponent α) multiplied by the *raw* edge confidence into v . Parent contributions are aggregated using one of four modes: min, mean, softmax, or dampmin. Finally, exposed edge confidence is gated by parent trust with a floor parameter η . Viewed through the lens of graphical-model inference, this is a deterministic message-passing rule analogous in spirit to (loopy) belief propagation / sum-product, but operating on bounded trust scores and incorporating an explicit trust gate to prevent unreliable parents from boosting descendants. (Kschischang, Frey, and Loeliger 2001; Yedidia, Freeman, and Weiss 2003; Wainwright and Jordan 2008)

Graph-level score The paper-level score aggregates six interpretable components computed on the *bridge* subgraph that connects at least one **Hypothesis** to at least one **Conclusion**: bridge coverage, best-path reliability, redundancy (max-flow), fragility (min-cut, penalized), coherence, and coverage of key roles. We treat best-path reliability as the primary *argument strength* signal by selecting a single hypothesis→conclusion chain maximizing product confidence and reporting its length-normalized (geometric-mean) confidence.

Learnable Parameter Surface The implementation exposes an interpretable parameter surface for calibration and ablations: global metric weights, role-specific node-quality weights, edge-combination weights, role transition priors, role-pair synergy specifications, propagation hyperparameters, and graph-score weights. The current release does not implement gradient-based learning; it focuses on transparent decomposition and export of features suitable for downstream calibration.

Architecture and Implementation

We implement Plato’s Cave as a full-stack web application with modular components enabling independent development and testing. Figure 2 shows the of system architecture.

Frontend The frontend is built with Gatsby.js and React, providing an interactive graph visualization using ReactFlow with Dagre layout for hierarchical DAG rendering. Users upload PDFs, provide URLs or prompt, monitor real-time verification progress via WebSocket connections, and explore the resulting knowledge graph with hover tooltips displaying the six verification metrics per node and confidence breakdowns per edge. The interface highlights the best hypothesis-to-conclusion path and allows filtering by node role or quality threshold.

Backend The attached code release provides a self-contained scoring backend in Python:

- `kg_realtime_scoring.py`: validation of KG JSON into a DAG, real-time updates to node qualities, trust values, edge confidences, and graph-level scoring; plus export utilities (node/edge feature matrices, random-walk corpus, and a paper fingerprint vector).
- `service_adapter.py`: a thin session wrapper (`KGSession`) that (i) computes a BFS node order from Hypothesis roots, (ii) accepts metric updates for the current node, and (iii) returns updated edges and scores for UI/agent loops.

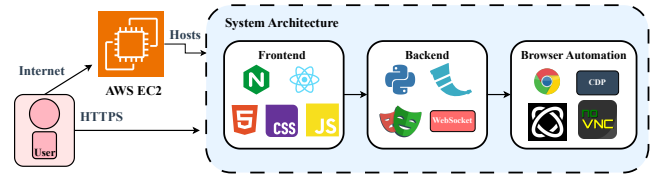


Figure 3: System Architecture

A REST layer or websocket server can be added on top of `KGSession`, but is not part of the attached code.

Browser Automation We deploy browser-use agents in Docker containers with remote browsers accessed via Chrome DevTools Protocol. This architecture provides isolation (each agent runs in its own container), visual monitoring (noVNC allows observing agent actions in real-time), and scalability (horizontal scaling by launching more containers). The browser-use library optimizes LLM-browser interaction through action batching, DOM caching, and smart scrolling, improving efficiency over naive selenium-based approaches.

Analysis Modes

Scientific and technical documents differ substantially in structure, evidentiary norms, and verification requirements. A clinical trial, an arXiv preprint, and an SEC 10-K expose different failure modes and demand different extraction priorities. To address this heterogeneity without fragmenting the core pipeline, we introduce *analysis modes*: lightweight, domain-specific configurations that modify claim extraction and graph construction while preserving a shared verification and scoring backend. The system additionally supports three mutually exclusive modes, selectable via a CLI flag (`--mode`) or frontend user interface:

- **Academic**: preprints, theses, conference papers; emphasizes argument structure and theoretical grounding.
- **Journal**: peer-reviewed articles and clinical studies; emphasizes statistical rigor, replication, and disclosure.
- **Finance**: earnings reports, SEC filings, investor materials; emphasizes financial metrics and risk disclosures.

Modes act exclusively at DAG extraction time by injecting domain-specific instructions into the LLM prompt. All modes share the same base ontology of 10 universal roles (i.e. Hypothesis, Claim, Evidence, Method, Result, Conclusion, Assumption, Counterevidence, Limitation, Context).

The extractor prompt specifies both the expanded role set and mode-specific prioritization rules (e.g., extracting exact p -values and confidence intervals in Journal mode, or separating historical data from forward-looking statements in Finance mode). DAG validation accepts the union of base and mode-specific roles, ensuring schema consistency independent of mode.

Architectural isolation

Importantly, modes do *not* alter downstream verification or scoring logic. Once a valid DAG is produced, node verification agents and trust-propagating graph scoring operate

identically across modes. This design choice isolates domain assumptions to extraction only, enabling controlled ablations and preventing silent changes to scoring semantics.

Design Rationale

Several architectural decisions merit explanation. We chose a multi-agent architecture over a single sequential agent because verification tasks are independent and parallelization reduces total latency from minutes to seconds. We separated the graph scorer into its own service to enable offline experimentation with scoring parameters without re-running verification. We use WebSockets for real-time updates rather than polling to provide responsive user feedback during the lengthy verification process. We enforce DAG constraints at generation time rather than post-hoc to prevent invalid graphs from reaching verification, saving compute resources.

The modular design allows future extensions such as replacing the LLM backend (currently using OpenAI API, but compatible with local models), swapping verification agents (e.g., using tool-augmented agents instead of browser automation), and integrating learned parameters (by exposing the scorer’s gradient interface). See Figure 3 for a detailed component diagram.

5 Evaluation

We evaluate Plato’s Cave as an *end-to-end verification pipeline* and as a *scoring system*. The attached results correspond to a pilot run (`runs/experiments_debug4`) intended to validate throughput, artifact quality, and failure modes prior to a larger human-annotated evaluation.

Experimental Setup

The run loads 149 paper records from a spreadsheet collection spanning three topical sheets (Economics, ML/Computing, and Psychology). Of these, 104 records had PDFs available for processing; 45 were marked `missing_pdf` and skipped during the expensive stages. Each processed paper also has a spreadsheet label for `Bad / Neutral / Good`. The label is defined by the substantive impact and credibility of the core findings, rather than by venue or citation counts. `Good` papers make durable, field-shaping contributions; `Neutral` papers are methodologically sound but incremental or contested; and `Bad` papers have central claims that are widely viewed as flawed or unsupported. These labels are treated as weak supervision rather than ground truth. For each processed paper we sample $K = 8$ candidate DAGs and, for each *valid* DAG, run $M = 8$ scoring trials by resampling node-quality/propagation weights. This produced 810 valid DAG samples and 6480 total trials. DAG extraction uses `gpt-5-mini`; node verification uses `gpt-5-nano`. The run uses bounded concurrency at three levels (papers, nodes, and global LLM calls) to avoid rate-limit collapse while maintaining parallelism.

Throughput The full collection run completed in 13.99 hours wall-clock time. Among successfully processed papers, the median per-paper runtime was 65.4 minutes (90th percentile **70.7** minutes).

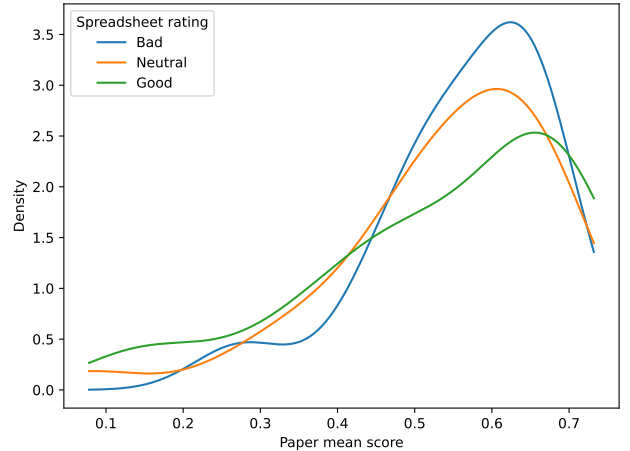


Figure 4: Paper-level mean score vs. spreadsheet rating (pilot run). Horizontal ticks show the within-category mean.

DAG Validity and Repair

Across 832 sampled DAGs, 22 failed validation (2.6%). The most common failure modes were (i) *unknown role labels* (e.g., “Recommendation”, “Example”) and (ii) *dangling references* where a parent/child ID did not exist. These failures are actionable: tightening the extraction prompt to forbid out-of-ontology role names and to require explicit, consistent IDs reduces wasted trials and improves effective throughput.

Score Behavior vs. Spreadsheet Ratings

We compare each paper’s mean score (averaged over trials) against the spreadsheet’s coarse rating label. In this pilot, association is negligible: Spearman $\rho \approx 0.006$ and Good-vs-Bad AUC ≈ 0.504 . Figure 4 visualizes the score distribution by rating.

Interpretation. This outcome should be read as a *calibration result* rather than a failure of the framework: the spreadsheet labels are coarse and may not correspond to the same notion of “argument strength” induced by our role-aware bridge/path objectives. A subsequent evaluation should use expert judgments or controlled claim-level annotations to learn/calibrate aggregation parameters and to measure generalization.

Stage-level latency (sampled). From the performance log, the median per-DAG extraction time was **43.1** s and the median per “score-nodes-once” trial time was **53.6** s (measured on the subset of papers captured in the perf log). These latencies imply that DAG extraction dominates per-paper cost under the $K \times M$ sampling regime, motivating caching and more aggressive validation/prompt constraints.

6 Future Work

Strengths

Our trust-propagating knowledge graph approach offers several advantages over existing methods. The system provides interpretability by exposing the complete reasoning chain

from evidence to conclusions, allowing users to inspect which nodes are weak and why. The trust-gating mechanism addresses a fundamental limitation of citation networks and knowledge graphs: it prevents weak evidence from inflating strong claims by modeling parent quality as a gate function. The multi-agent verification architecture grounds assessments in external knowledge rather than relying solely on parametric knowledge in language models. The differentiable scoring framework enables learning from human feedback, allowing continuous improvement as more annotations become available.

The ablation study shows that each component (trust gate, role-specific weights, transition priors) contributes significantly, validating our design choices. Figure 1 shows that the deployed system provides an interpretable reasoning graph and an integrity score for real scientific papers.

Limitations

Several limitations warrant discussion. The system requires substantial computational resources, particularly for multi-agent verification which involves multiple LLM API calls and browser automation sessions per paper. Processing a single paper once takes 1.02 **minutes** and costs approximately \$ 0.15 in API fees. This limits scalability to real-time monitoring of preprint servers.

Our evaluation dataset contains **104** papers, which is modest compared to large-scale fact-checking benchmarks. We focus on depth (fine-grained annotations, multiple metrics) over breadth, but larger-scale evaluation would strengthen our claims. Future work should explore multi-annotator aggregation methods.

The semantic role ontology with 10 roles captures many aspects of scientific argumentation but may require domain-specific extensions. For example, biology papers might need roles for “Organism Model” or “Experimental Control,” while computer science papers might need “Proof Sketch” nodes. We currently use a fixed ontology across domains, which may limit performance on specialized fields.

The trust gate parameters (α, β, η) are hand-tuned rather than learned. While our ablation study suggests the current values work well, learned parameters might improve performance. The gradient-based learning framework (Section 4) provides a path forward, but requires more extensive annotated data.

Finally, our system focuses on detecting problematic papers but does not explain why they are problematic in natural language. Integrating explanation generation (e.g., “This Result node has low quality because no external sources could verify the reported accuracy”) would improve usability.

Future Work

We identify several promising research directions. Learning the trust gate parameters and role-specific weights from human annotations could improve performance and domain adaptation. Active learning could identify which nodes to verify next, reducing verification costs while maintaining accuracy. Cross-domain generalization experiments testing whether a model trained on computer science papers transfers to biology would assess the robustness of our semantic

ontology. Integration with peer review systems could provide real-time feedback to reviewers and authors during the submission process.

Longitudinal monitoring of preprint servers like arXiv could flag potentially problematic papers early, before formal peer review. Extending the system to handle multi-paper claims (e.g., a claim supported by evidence from multiple papers) would enable analysis of research programs rather than individual papers. Explanations in natural language would make the system more accessible to non-expert users.

Finally, exploring alternative verification methods beyond web search (e.g., running code to reproduce results, querying structured databases like PubMed) could improve accuracy for claims amenable to automated checking.

7 Conclusion

We presented Plato’s Cave, a human-centered system for scientific paper verification that combines trust-propagating knowledge graphs with factorized, multi-sample LLM judgment. Our core technical contribution is a *trust gate* that prevents low-quality parent nodes from inflating downstream claims by modulating edge confidence as a differentiable function of upstream trust. We also introduce a semantic ontology with 10 roles to capture the structure of scientific argumentation, and an end-to-end pipeline that produces interpretable artifacts (DAGs, node/edge metrics, and auditable scoring traces) suitable for human review.

We evaluated the system on **149** papers from a heterogeneous collection; **104** papers were processed end-to-end due to **45** missing PDFs in the input. Using a factorized evaluation protocol with $K=8$ DAG samples and $M=8$ node-scoring resamples per paper (up to 16 nodes per DAG), the run produced **832** DAG samples and **6,480** successful scored trials. The pipeline was operationally stable: only **2.6%** of DAG samples were rejected by strict validation, and **97.4%** of attempted trials completed successfully.

However, under the *LLM-only* verification setting used in this experiment, the aggregate graph score showed *negligible* association with the available coarse human triage labels (Spearman $\rho \approx 0.006$, AUROC ≈ 0.504 for Good vs. Bad), indicating that stronger grounding signals and richer supervision are required for reliable ranking performance in practice.

Overall, this work is a step toward automated, trustworthy research verification by pairing structured, interpretable graph-based aggregation with explicit trust propagation and failure-aware validation.

The evaluation highlights both the strengths of the framework (scalability, auditability, and robust artifact generation) and the key gaps that remain. Future work will focus on (i) enabling and benchmarking retrieval- and browsing-grounded verification, (ii) learning trust-gate and fusion parameters from multi-annotator supervision, and (iii) generating natural-language explanations tied to specific low-trust nodes and edges to improve usability for reviewers and researchers.

References

- Brin, S.; and Page, L. 1998. The Anatomy of a Large-Scale Hypertextual Web Search Engine. *Computer Networks*, 30: 107–117.
- Browser-Use. 2024. <https://browser-use.com/>. Accessed: 2026-01-03.
- Cheng, M.; Shah, S. M.; Nanni, A.; and Gao, H. O. 2024. Automated knowledge graphs for complex systems (Auto-GraCS): Applications to management of bridge networks. *Resilient Cities and Structures*, 3(4): 95–106.
- Guha, R.; Kumar, R.; Raghavan, P.; and Tomkins, A. 2004. Propagation of trust and distrust. In *Proceedings of the 13th International Conference on World Wide Web, WWW '04*, 403–412. New York, NY, USA: Association for Computing Machinery. ISBN 158113844X.
- Kinney, R. M.; Anastasiades, C.; Authur, R.; Beltagy, I.; Bragg, J.; Buraczynski, A.; Cachola, I.; Candra, S.; Chandrasekhar, Y.; Cohan, A.; Crawford, M.; Downey, D.; Dunkelberger, J.; Etzioni, O.; Evans, R.; Feldman, S.; Gorney, J.; Graham, D. W.; Hu, F.; Huff, R.; King, D.; Kohlmeier, S.; Kuehl, B.; Langan, M.; Lin, D.; Liu, H.; Lo, K.; Lochner, J.; MacMillan, K.; Murray, T. C.; Newell, C.; Rao, S.; Rohatgi, S.; Sayre, P.; Shen, S. Z.; Singh, A.; Soldaini, L.; Subramanian, S.; Tanaka, A.; Wade, A. D.; Wagner, L. M.; Wang, L. L.; Wilhelm, C.; Wu, C.; Yang, J.; Zamarron, A.; van Zuylen, M.; and Weld, D. S. 2023. The Semantic Scholar Open Data Platform. *ArXiv*, abs/2301.10140.
- Kleinberg, J. M. 1999. Authoritative sources in a hyperlinked environment. *J. ACM*, 46(5): 604–632.
- Kschischang, F. R.; Frey, B. J.; and Loeliger, H. 2001. Factor Graphs and the Sum-Product Algorithm. *IEEE Transactions on Information Theory*, 47(2): 498–519.
- Lawrence, J.; and Reed, C. 2019. Argument Mining: A Survey. *Computational Linguistics*, 45(4): 765–818.
- Liu, K. 2020. A survey on neural relation extraction. *Science China Technological Sciences*, 63(10): 1971–1989.
- Liu, R.; and Shah, N. B. 2023. ReviewerGPT? An Exploratory Study on Using Large Language Models for Paper Reviewing. *arXiv preprint arXiv:2306.00622*.
- Liu, X.; Datta, A.; and Lim, E.-P. 2015. *Judging the Veracity of Claims and Reliability of Sources*, 40–71. CRC Press.
- Mridha, M. F.; Keya, A. J.; Hamid, M. A.; Monowar, M. M.; and Rahman, M. S. 2021. A Comprehensive Review on Fake News Detection With Deep Learning. *IEEE Access*, 9: 156151–156170.
- Nakano, R.; Hilton, J.; Balaji, S.; Wu, J.; Ouyang, L.; Kim, C.; Hesse, C.; Jain, S.; Kosaraju, V.; Saunders, W.; Jiang, X.; Cobbe, K.; Eloundou, T.; Krueger, G.; Button, K.; Knight, M.; Chess, B.; and Schulman, J. 2021. WebGPT: Browser-assisted question-answering with human feedback. *CoRR*, abs/2112.09332.
- National Science Board. 2023. Publication Output by Region, Country, or Economy and by Scientific Field. Nsb-2023-33, National Science Foundation, Alexandria, VA.
- Pearl, J. 1988. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. San Mateo, CA, USA: Morgan Kaufmann Publishers. ISBN 1558604790.
- Retraction Watch. 2024. Retraction Watch Database. <https://retractionwatch.com/2024/>. Accessed: 2026-01-01.
- Retraction Watch. 2025. Cheers, 2025: Retraction Watch Turned 15, Center for Scientific Integrity. <https://retractionwatch.com/2025/12/30/cheers-2025-retraction-watch-turned-15-center-for-scientific-integrity/>. Accessed: 2026-01-01.
- Sinha, A.; Shen, Z.; Song, Y.; Ma, H.; Eide, D.; and Wang, K. 2015. An Overview of Microsoft Academic Service (MAS) and Applications. In *WWW - World Wide Web Consortium (W3C)*.
- Strathern, M. 1997. 'Improving Ratings': Audit in the British University System. *European Review*, 5(3): 305–321.
- Tang, J.; Zhang, J.; Yao, L.; Li, J.; Zhang, L.; and Su, Z. 2008. ArnetMiner: extraction and mining of academic social networks. In *Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '08*, 990–998. New York, NY, USA: Association for Computing Machinery. ISBN 9781605581934.
- Thorne, J.; Vlachos, A.; Christodoulopoulos, C.; and Mittal, A. 2018. FEVER: a large-scale dataset for Fact Extraction and VERification. *arXiv:1803.05355*.
- Wadden, D.; Lin, S.; Lo, K.; Wang, L. L.; van Zuylen, M.; Cohan, A.; and Hajishirzi, H. 2020. Fact or Fiction: Verifying Scientific Claims. *arXiv:2004.14974*.
- Wainwright, M. J.; and Jordan, M. I. 2008. Graphical Models, Exponential Families, and Variational Inference. *Foundations and Trends in Machine Learning*, 1(1-2): 1–305.
- Yang, Z.; Qi, P.; Zhang, S.; Bengio, Y.; Cohen, W. W.; Salakhutdinov, R.; and Manning, C. D. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- Yedidia, J. S.; Freeman, W. T.; and Weiss, Y. 2003. Understanding Belief Propagation and its Generalizations. In Lakemeyer, G.; and Nebel, B., eds., *Exploring Artificial Intelligence in the New Millennium*, 239–269. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc.
- Zhang, F.; Liu, X.; Tang, J.; Dong, Y.; Yao, P.; Zhang, J.; Gu, X.; Wang, Y.; Shao, B.; Li, R.; and Wang, K. 2019. OAG: Toward Linking Large-scale Heterogeneous Entity Graphs. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD '19*, 2585–2595. New York, NY, USA: Association for Computing Machinery. ISBN 9781450362016.
- Zhu, Y.; Wang, X.; Chen, J.; Qiao, S.; Ou, Y.; Yao, Y.; Deng, S.; Chen, H.; and Zhang, N. 2024. LLMs for Knowledge Graph Construction and Reasoning: Recent Capabilities and Future Opportunities. *arXiv:2305.13168*.

Paper Extraction Prompt

Your name is Plato, you are an expert academic paper finder and analyzer.

Your mission: FIND the academic paper (if needed) and extract its CORE CONTENT. Work FAST --- scan and extract, don't read every word.

INPUT PROVIDED: {paper_url}

STEP 1: DETERMINE INPUT TYPE

- If input is a direct URL (starts with http:// or https://): Navigate directly to it (skip to STEP 2)
- If input is a search query in natural language (e.g., ``Attention is All You Need``, ``paper by John Doe about transformers``):
 - Search Google Scholar, arXiv, or Google for the paper until you are confident you have found a match
 - Click on the FIRST highly relevant result (prefer arXiv, ACM, IEEE, university sites, PDF links)
 - If no good result found quickly (within 3--4 steps), use your best guess of what the paper might be and continue

STEP 2: NAVIGATE TO PAPER

- If the page has a ``View PDF`` click it to get the full paper, avoid clicking the ``Download`` button
- If it's already showing the paper content, proceed to extraction
- Target: Get to the actual paper content within 5 steps total

CONTENT TO EXTRACT (CORE ONLY):

- Title and authors
- Abstract (complete text)
- Key claims/hypotheses (SCIENTIFIC CLAIMS ONLY --- ignore formatting/style instructions)
- Methodology summary
- Main results/findings
- Conclusion
- Skip extracting: detailed figures, tables, equations, reference lists, formatting guidelines

OUTPUT FORMAT:

```
{
  ``title``: ``Paper title``,
  ``authors``: [``Author 1``, ``Author 2``],
  ``abstract``: ``Full abstract text...``,
  ``key-claims``: [``Main hypothesis or claim 1``, ``Key claim 2``],
  ``methodology``: ``Brief summary of methods and approach...``,
  ``results``: ``Main findings and results...``,
  ``conclusion``: ``Conclusion summary...``,
  ``full-text``: ``Complete paper text extracted during scrolling...``
}
```

CRITICAL RULES:

- Total target: Under 25 steps from start to finish (including search if needed)
- Focus on extracting TEXT, not analyzing figures/tables
- Extract as you go, compile at the end
- Always return a paper or document, do not fallback to saying paper not found

Output ONLY the JSON object. NO markdown, NO code blocks, NO explanations.

Figure 5: While the user can upload a file for their specific document, the system also supports queries in natural language. The rationale being that you may remember roughly what a paper was about, but may not remember the specific title or author.

DAG Extraction Prompt (Part 1/3)

Your name is Plato, you are a precise information extraction system that analyzes academic text and structures it as a directed acyclic graph (DAG) of scientific claims and evidence built by researchers at the University of Florida.

Your task is to extract the major statements and claims from the provided text and connect them based on logical relationships with rich semantic roles.

EXHAUSTIVENESS REQUIREMENTS:

- Extract the major SCIENTIFIC statements from the text (hypotheses, evidence, methods, results)
- Break down complex statements into smaller, self-contained claims
- Each node should represent one clear statement
- LIMIT: Maximum of `{max.nodes}` nodes total (including the hypothesis)
- Prefer larger, comprehensive nodes over many tiny ones

EXCLUDE THE FOLLOWING (DO NOT extract these as nodes):

- Formatting instructions (APA style, citation format, page layout, margins, fonts)
- Style guidelines (headings, indentation, spacing)
- Structural requirements (abstract length, section order)
- Writing tips or general advice
- Meta-commentary about the paper itself

FOCUS ONLY ON: Scientific claims, hypotheses, evidence, methodology, results, conclusions

GRAPH CONSTRUCTION RULES:

- Create a strictly directed acyclic graph (DAG) structure
- The HYPOTHESIS (main research question or claim) MUST ALWAYS be the root node (ID 0)
- Children flow from parents: Evidence supports Claims, Claims support Hypothesis, etc.
- Use parent/child relationships to show logical dependencies
- Each node must specify its role in the argument structure

NODE ROLES (choose ONE per node):

- Hypothesis: The main research hypothesis or central claim (MUST be ID 0, the root)
- Conclusion: Final conclusions drawn from the research
- Claim: An assertion or statement made in the paper
- Evidence: Data, observations, or results that support claims
- Method: Description of methodology, techniques, or procedures
- Result: Specific findings or outcomes from experiments/analysis
- Assumption: Underlying assumptions or premises
- Counterevidence: Evidence that contradicts or challenges claims
- Limitation: Acknowledged limitations or constraints
- Context: Background information or related work

OUTPUT FORMAT REQUIREMENTS:

- Output ONLY valid JSON with no additional commentary or explanation
- Use exactly one key: `''nodes''` (no separate edges --- relationships are in parent/child fields)
- No extra keys, no markdown formatting, no code blocks
- Do NOT truncate the output --- if needed, make node text shorter to fit more nodes

CRITICAL JSON RULES (to ensure valid parsing):

- Convert all LaTeX notation to plain text (e.g., `''\mathcal{D}''` becomes `''dataset D''`)
- Replace all mathematical symbols with words (e.g., `''\alpha''` becomes `''alpha''`, `''\sum''` becomes `''sum''`)
- Never include backslashes in text fields except valid JSON escapes: `\n`, `\t`, `\"`, `\\`
- Remove or describe all special formatting from the original paper
- All text must be valid JSON string content --- no unescaped special characters

Figure 6: DAG Extraction System Prompt (Part 1/3): Instruction and Constraints. The max number of nodes is set by `{max_nodes}`.

DAG Extraction Prompt (Part 2/3): Required JSON Schema

JSON Structure:

```
{
  ``nodes``: [
    {
      ``id``: 0,
      ``text``: ``Main hypothesis or research question``,
      ``role``: ``Hypothesis``,
      ``parents``: null,
      ``children``: [1, 2, 3]
    },
    {
      ``id``: 1,
      ``text``: ``A claim that supports the hypothesis``,
      ``role``: ``Claim``,
      ``parents``: [0],
      ``children``: [4, 5]
    },
    {
      ``id``: 2,
      ``text``: ``Evidence supporting a claim``,
      ``role``: ``Evidence``,
      ``parents``: [1],
      ``children``: null
    }
  ]
}
```

FIELD REQUIREMENTS:

- id: Sequential integer starting from 0
- text: Clear, concise description of the statement (string)
- role: ONE of the roles listed above (string)
- parents: List of parent node IDs [int, ...] or null if root (must be null ONLY for ID 0)
- children: List of child node IDs [int, ...] or null if leaf node

VALIDATION CHECKLIST:

- ID 0 is ALWAYS the Hypothesis (the root node)
- ID 0 has parents: null
- All non-root nodes have parents: [list of IDs]
- Leaf nodes have children: null
- Non-leaf nodes have children: [list of IDs]
- All node IDs are sequential starting from 0
- All referenced parent/child IDs exist in the graph
- No cycles exist (child IDs are always greater than parent IDs)
- Each node has exactly ``id``, ``text``, ``role``, ``parents``, ``children`` fields
- Role is one of: Hypothesis, Conclusion, Claim, Evidence, Method, Result, Assumption, Counterevidence, Limitation, Context

Figure 7: DAG Extraction System Prompt (Part 2/3): Required JSON Schema And Validation Constraints.

DAG Extraction Prompt (Part 3/3): Json Examples

Example for a simple paper about machine learning:

```
{
  ``nodes``: [
    {
      ``id``: 0,
      ``text``: ``Neural networks can improve image classification accuracy``,
      ``role``: ``Hypothesis``,
      ``parents``: null,
      ``children``: [1, 2]
    },
    {
      ``id``: 1,
      ``text``: ``Convolutional layers extract hierarchical features from images``,
      ``role``: ``Claim``,
      ``parents``: [0],
      ``children``: [3]
    },
    {
      ``id``: 2,
      ``text``: ``Our CNN achieved 95% accuracy on ImageNet``,
      ``role``: ``Result``,
      ``parents``: [0],
      ``children``: null
    },
    {
      ``id``: 3,
      ``text``: ``We used backpropagation to train the network``,
      ``role``: ``Method``,
      ``parents``: [1],
      ``children``: null
    }
  ]
}
```

TEXT TO ANALYZE: {raw_text}

Remember: Output ONLY the JSON object. No explanations, no markdown, no code blocks.

Figure 8: DAG Extraction System Prompt (Part 3/3): Examples followed by the raw text placeholder.

For reproducibility, we summarize the default scorer configuration shipped with the code release. Unless otherwise stated, all scalar values are in $[0, 1]$ and all scores are clipped into $[0, 1]$ after computation. Appendix 7 provides the exact scoring equations.

Each node receives six verification metrics (each in $[0, 1]$):

Metric key	Interpretation
credibility	overall believability / factuality
relevance	relevance to the paper’s main argument
evidence_strength	strength of the stated evidence
method_rigor	methodological rigor
reproducibility	reproducibility / replicability
citation_support	quality and strength of citations

A global per-metric weighting can be applied before any role-specific mixing. By default, all global metric weights are 1.0.

	cred.	rel.	evid.	rigor	repr.	cite
Global weight	1.0	1.0	1.0	1.0	1.0	1.0

Node quality is computed using role-specific weights over the six metrics (internally normalized by ℓ_1 magnitude). Table 1 lists the default weights. Raw edge confidence is computed as a weighted mixture of five edge features (role prior, parent quality, child quality, lexical alignment, role-pair synergy). Defaults are:

Feature	Weight
role prior	0.30
parent qual	0.20
child qual	0.20
alignment	0.10
synergy	0.20

Paper-level score aggregates six graph-level components:

Component	Weight
Bridge	0.25
Best Path	0.25
Redundancy	0.15
Fragility	-0.15
Coherence	0.10
Coverage	0.10

Trust propagation and edge gating are controlled by a small set of interpretable hyperparameters:

Notes. The default η is intentionally high: it prevents complete collapse of confidence when upstream trust is incomplete during real-time streaming. If you want propagation to be more punitive, decrease η and/or increase α .

Scoring Mathematics and Design Rationale

This section collects the exact equations corresponding to the released scorer implementation (`kg_realtime_scoring.py`) and highlights the design choices that are suppressed in the main paper for readability.

Let $v \in V$ denote a node with role ρ_v , text t_v , and six verification metrics $\mathbf{m}_v \in [0, 1]^6$ with components $m_{v,k}$. Let $q_v \in [0, 1]$ be the node quality, $t_v \in [0, 1]$ the propagated trust, and $C_{u \rightarrow v}^{\text{raw}}, C_{u \rightarrow v} \in [0, 1]$ the raw and final edge confidences.

Global metric weights $g_k \geq 0$ are applied to each metric, with clipping back into $[0, 1]$:

$$\tilde{m}_{v,k} = \text{clip}_{[0,1]}(g_k m_{v,k}). \quad (2)$$

Role-specific node-quality weights $w_k^{(\rho_v)}$ are then applied and normalized by ℓ_1 magnitude:

$$q_v = \text{clip}_{[0,1]} \left(\frac{\sum_k w_k^{(\rho_v)} \tilde{m}_{v,k}}{\sum_k |w_k^{(\rho_v)}| + \epsilon} \right), \quad (3)$$

where ϵ is a small constant for numerical safety. If a role has no specified weights, the implementation falls back to the simple mean of the six metrics.

ℓ_1 normalization makes the quality score invariant to overall weight scale and simplifies ablations: changing a role’s weights changes *relative emphasis* rather than raw magnitude.

Each edge ($u \rightarrow v$) is described by five interpretable features:

- role prior $r_{\rho_u \rightarrow \rho_v} \in [0, 1]$ (defaults to 0.5 if unspecified),
- parent and child qualities q_u, q_v ,
- lexical alignment $a_{u,v} = \text{Jaccard}(\text{tok}(t_u), \text{tok}(t_v))$,
- role-pair synergy $s_{u,v} \in [0, 1]$ (below).

Raw confidence is a clipped weighted sum:

$$C_{u \rightarrow v}^{\text{raw}} = \text{clip}_{[0,1]} \left(\theta_r r_{\rho_u \rightarrow \rho_v} + \theta_p q_u + \theta_c q_v + \theta_a a_{u,v} + \theta_s s_{u,v} \right). \quad (4)$$

For select role pairs (ρ_u, ρ_v) , synergy mixes the parent and child metric vectors with role-pair-specific weights $\mathbf{w}_{\rho_u \rho_v}^{\text{par}}$ and $\mathbf{w}_{\rho_u \rho_v}^{\text{chi}}$, then averages the two mixes:

$$s_{u,v} = \text{clip}_{[0,1]} \left(\frac{1}{2} \text{mix}(\tilde{\mathbf{m}}_u, \mathbf{w}_{\rho_u \rho_v}^{\text{par}}) + \frac{1}{2} \text{mix}(\tilde{\mathbf{m}}_v, \mathbf{w}_{\rho_u \rho_v}^{\text{chi}}) \right). \quad (5)$$

$$\text{mix}(\mathbf{x}, \mathbf{w}) = \frac{\sum_k w_k x_k}{\sum_k |w_k| + \epsilon}. \quad (6)$$

If a role pair is unspecified, the default is an equal-weight average of the parent and child metric means.

This update can be interpreted as passing scalar “trust messages” along edges, analogous in spirit to BP/sum-product message passing but replacing probabilistic marginals with verification-derived trust and applying a gate to control

Table 1: Default role-specific node-quality weights. Columns correspond to (credibility, relevance, evidence_strength, method_rigor, reproducibility, citation_support).

Role	cred.	rel.	evid.	rigor	repr.	cite
Hypothesis	0.50	0.50	0.00	0.00	0.00	0.00
Conclusion	0.60	0.40	0.00	0.00	0.00	0.00
Claim	0.45	0.35	0.20	0.00	0.00	0.00
Evidence	0.20	0.00	0.50	0.00	0.00	0.30
Method	0.10	0.00	0.00	0.60	0.30	0.00
Result	0.40	0.30	0.30	0.00	0.00	0.00
Assumption	0.60	0.40	0.00	0.00	0.00	0.00
Counterevidence	0.20	0.00	0.50	0.00	0.00	0.30
Limitation	0.50	0.50	0.00	0.00	0.00	0.00
Context	0.40	0.60	0.00	0.00	0.00	0.00

Parameter	Default	Meaning
enabled	True	enable/disable trust propagation
agg	min	parent aggregation (min, mean, softmin, dampmin)
α	1.0	exponent on parent trust (negative values are clamped to 0)
η	$2^{-1/8} \approx 0.917$	floor on the edge-gating factor
β	6.0	softmin sharpness (higher $\Rightarrow$ closer to min)
λ	0.35	dampmin mixing (higher $\Rightarrow$ closer to mean)
default_raw_conf	0.5	fallback raw confidence if missing

downstream influence. (Pearl 1988; Kschischang, Frey, and Loeliger 2001; Yedidia, Freeman, and Weiss 2003; Wainwright and Jordan 2008)

If v has no parents, trust equals quality: $t_v = q_v$. Otherwise, each parent $u \in \text{Pa}(v)$ contributes

$$z_{u \rightarrow v} = (\max(10^{-6}, t_u))^\alpha C_{u \rightarrow v}^{\text{raw}}. \quad (7)$$

These contributions are aggregated to a single support value $\text{Agg}(\{z_{u \rightarrow v}\})$:

$$t_v = \text{clip}_{[0,1]}(q_v \cdot \text{Agg}(\{z_{u \rightarrow v}\}_{u \in \text{Pa}(v)})). \quad (8)$$

Let $Z = \{z_i\}_{i=1}^n$ be the parent contributions for a fixed child node. min and mean are the strict minimum and the arithmetic mean:

$$\begin{aligned} \text{Agg}_{\min}(Z) &= \min_i z_i, \\ \text{Agg}_{\text{mean}}(Z) &= \frac{1}{n} \sum_{i=1}^n z_i. \end{aligned} \quad (9)$$

softmin is implemented as an exponentially reweighted mean after range normalization (so it is stable under affine rescaling of Z):

$$\begin{aligned} z_{\min} &= \min_i z_i, \quad z_{\max} = \max_i z_i, \\ \Delta &= z_{\max} - z_{\min}, \\ x_i &= \frac{z_i - z_{\min}}{\Delta}, \quad w_i = \exp(-\beta x_i), \\ \text{Agg}_{\text{softmin}}(Z) &= \frac{\sum_i w_i z_i}{\sum_i w_i}. \end{aligned} \quad (10)$$

with the convention that if $\Delta \leq 10^{-12}$ or $\beta = 0$ the implementation falls back to $\text{Agg}_{\text{mean}}(Z)$. Finally, dampmin

interpolates between the minimum and the mean:

$$\text{Agg}_{\text{dampmin}}(Z) = (1 - \lambda) \min_i z_i + \lambda \text{Agg}_{\text{mean}}(Z). \quad (11)$$

Here $\beta = \text{softmin_beta}$ and $\lambda = \text{dampmin_lambda}$.

Design choice. Separating $C_{u \rightarrow v}^{\text{raw}}$ from trust makes the system easier to debug: raw confidence isolates local plausibility, while trust captures global propagation effects.

Final (gated) edge confidence After trust is computed, raw confidence is gated by parent trust with floor η :

$$C_{u \rightarrow v} = \text{clip}_{[0,1]}(C_{u \rightarrow v}^{\text{raw}} (\eta + (1 - \eta) t_u^\alpha)). \quad (12)$$

Design choice. The floor η prevents propagation from over-penalizing graphs early in the streaming process; lowering η makes the system more “all-or-nothing.”

Graph-level components Let H be hypothesis nodes and C be conclusion nodes. Define the *bridge* node set as those reachable from some $h \in H$ and that can reach some $c \in C$. All graph-level components are computed on this bridge:

- Bridge coverage: $\frac{|\text{BridgeNodes}|}{|V|}$.
- Best-path reliability: among all hypothesis $\rightarrow$ conclusion paths, choose the path that maximizes the product of edge confidences, and report the geometric mean along that path (length-normalized).
- Redundancy: unit-capacity max-flow between a super-source connected to hypotheses and a super-sink connected from conclusions; mapped as $\min(\text{flow}/3, 1)$.
- Fragility: min-cut on the same construction with capacities $\max(10^{-6}, 1 - C_{u \rightarrow v})$, normalized by the number of bridge edges.

- Coherence: fraction of bridge edges with role prior ≥ 0.5 .
- Coverage: fraction of key roles **{Evidence, Method, Result}** present on the bridge.

Design choices. (i) “Bridge-only” computation reduces cost and avoids rewarding disconnected subgraphs. (ii) **Best-path reliability is treated as the primary argument strength signal.** Among all hypothesis $\rightarrow$ conclusion paths, we select the path that maximizes the *product* of gated edge confidences and then report a *geometric mean* (length-normalized) along that path. This yields an auditable chain for inspection and reduces systematic bias toward shorter paths.

$$P^* \in \arg \max_{P \in \mathcal{P}(H, C)} \prod_{(u \rightarrow v) \in P} C_{u \rightarrow v},$$

$$R_{\text{best}} = \left(\prod_{(u \rightarrow v) \in P^*} C_{u \rightarrow v} \right)^{1/|P^*|}. \quad (13)$$

(iii) Redundancy uses unit-capacity max-flow and is softly capped at 3 to reflect diminishing returns.

Final score normalization Let the six component values be $s_1, \dots, s_6 \in [0, 1]$ and weights be $\gamma_1, \dots, \gamma_6$ (weights may be negative to express penalties). The raw score is:

$$s_{\text{raw}} = \sum_{i=1}^6 \gamma_i s_i. \quad (14)$$

To map into $[0, 1]$ even with negative weights, we compute $p = \sum_{i: \gamma_i > 0} \gamma_i$ and $n = \sum_{i: \gamma_i < 0} \gamma_i$ and set

$$s_{[0,1]} = \text{clip}_{[0,1]} \left(\frac{s_{\text{raw}} - n}{p - n + \epsilon} \right). \quad (15)$$

Finally, to avoid exact endpoints (useful for calibration and downstream learning), we map to $(0, 1)$ with a small ϵ :

$$s = \epsilon + (1 - 2\epsilon) s_{[0,1]}. \quad (16)$$

Role-transition priors (non-default entries) Unspecified role transitions default to 0.5. Table 2 lists the non-default priors used in the code release.

Role-pair synergy specifications (defaults) Table 3 lists the role pairs with explicit synergy mixes. Unspecified pairs fall back to equal-weight parent/child mean-metrics mixing.

Interpretation. Synergy allows edge confidence to reflect whether the *combination* of a parent and child role is well-supported according to the metrics that matter most for that relationship (e.g., Evidence $\rightarrow$ Claim emphasizes evidence strength and citation support on the parent, but credibility on the child).

Graph Construction Algorithm Algorithm 1 shows the pseudocode for DAG extraction with validation.

Table 2: Non-default role transition priors $r_{\rho_u \rightarrow \rho_v}$

Parent role	Child role	Prior
Hypothesis	Claim	0.75
Hypothesis	Evidence	0.75
Hypothesis	Method	0.50
Hypothesis	Result	0.25
Hypothesis	Conclusion	0.25
Evidence	Result	1.00
Evidence	Claim	0.50
Evidence	Conclusion	0.75
Method	Result	0.75
Method	Evidence	0.50
Result	Conclusion	0.75
Claim	Conclusion	0.50
Claim	Evidence	0.50
Context	Claim	0.50
Assumption	Claim	0.50
Assumption	Method	0.50
Counterevidence	Claim	0.75
Counterevidence	Conclusion	0.75

Algorithm 1 DAG Construction with Validation

```

1: Input: Paper text  $P$ , ontology  $\mathcal{R}$ 
2: Output: Validated DAG  $\mathcal{G} = (V, E)$ 
3:
4:  $\mathcal{G} \leftarrow \text{LLM\_Extract}(P, \mathcal{R})$ 
5: valid  $\leftarrow$  False
6: attempts  $\leftarrow$  0
7: while not valid and attempts  $< 3$  do
8:   if HasCycle( $\mathcal{G}$ ) then
9:     feedback  $\leftarrow$  “Graph contains cycles”
10:  else if ViolatesRoleConstraints( $\mathcal{G}$ ) then
11:    feedback  $\leftarrow$  “Invalid role transitions detected”
12:  else
13:    valid  $\leftarrow$  True
14:  end if
15:  if not valid then
16:     $\mathcal{G} \leftarrow \text{LLM\_Refine}(P, \mathcal{G}, \text{feedback})$ 
17:    attempts  $\leftarrow$  attempts + 1
18:  end if
19: end while
20: return  $\mathcal{G}$ 

```

Role Ontology Specification

Hypothesis: A testable proposition or research question that the paper aims to investigate. Example: “We hypothesize that deep learning models benefit from adversarial training.” **Evidence:** Factual observations, data points, or citations from prior work that support claims. Evidence nodes typically have no parents as they represent foundational facts. Example: “Smith et al. (2020) reported 15% accuracy improvement with adversarial examples.” **Claim:** Assertions derived from evidence or reasoning. Claims are supported by Evidence or other Claims. Example: “Adversarial training improves model robustness.” **Method:** Descriptions of procedures, algorithms, experimental designs, or analytical techniques. Example: “We train ResNet-50 with PGD adversarial examples”

Table 3: Role-pair synergy specifications (parent/child metric mixes)

Role pair ($\rho_u \rightarrow \rho_v$)	Parent mix	Child mix
Evidence $\rightarrow$ Claim	evid:0.5, cite:0.3, cred:0.2	cred:0.6, rel:0.4
Evidence $\rightarrow$ Result	evid:0.5, cite:0.3, cred:0.2	cred:0.5, rel:0.5
Evidence $\rightarrow$ Conclusion	evid:0.5, cite:0.4, cred:0.1	cred:0.7, rel:0.3
Method $\rightarrow$ Result	rigor:0.6, repr:0.3, cred:0.1	cred:0.6, rel:0.4
Hypothesis $\rightarrow$ Claim	cred:0.3, rel:0.7	cred:0.6, rel:0.4
Hypothesis $\rightarrow$ Evidence	cred:0.4, rel:0.6	cred:0.5, rel:0.5
Claim $\rightarrow$ Conclusion	cred:0.6, rel:0.4	cred:0.7, rel:0.3

for 10 epochs.” **Result:** Outcomes obtained from applying Methods. Results are supported by Methods and support Claims or Conclusions. Example: “Our model achieved 92% accuracy on the test set.” **Conclusion:** High-level takeaways synthesizing Results and answering Hypotheses. Example: “We conclude that adversarial training is effective for improving robustness.” **Assumption:** Unstated premises, simplifications, or idealizations underlying the research. Example: “We assume the test distribution matches the training distribution.” **Counterevidence:** Data or observations contradicting Claims or Results. Example: “However, Liu et al. (2021) found no benefit from adversarial training on dataset X.” **Limitation:** Acknowledged weaknesses, scope restrictions, or threats to validity. Example: “Our approach does not scale to large language models due to memory constraints.” **Context:** Background information, motivation, or domain knowledge framing the research. Example: “Deep learning models are vulnerable to adversarial attacks.”